

# DeepResearcher

## 完整研究：Scaling Deep Research via Real-World Reinforcement Learning

真实 Web Environment、GRPO、Browsing Multi-Agent、Search-dependent Data 与 2026 后续演化

Deep Research 训练范式演化：Local RAG RL → Real-Web RL → Long-form Research RL



研究基线：DeepResearcher (EMNLP 2025 / arXiv:2504.03160)、GAIR-NLP/DeepResearcher 官方仓库与 DeepResearcher-7B 模型卡；并结合 Search-R1、WebThinker、DR Tulu 与 2026 Search-R1++ 作为后续对照。  
研究日期：2026-08-13。

### 核心判断

DeepResearcher 最重要的贡献不是“又一个 Search Agent”，而是第一次把\*\*真实开放 Web 本身变成强化学习环境\*\*。它证明：Agent 的研究能力不仅由模型与奖励决定，\*\*Environment Fidelity（环境真实性）也是训练目标的一部分\*\*。

# 目录与执行摘要

1. 时代位置：从 Local RAG RL 到 Real-Web RL
2. 核心命题：Environment 是 Research Policy 的一部分
3. Deep Research Trajectory：Think / Search / Browse / Answer
4. web\_search：候选发现与 Query Policy
5. web\_browse：Reading Agents、Memory 与 Synthesis
6. 为什么浏览工具内部采用 Multi-Agent
7. 真实 Web RL 的三类工程难题
8. GRPO、Observation Masking 与训练目标
9. Reward：为什么采用 Format Penalty + F1
10. Search-dependent Data：80K 数据如何构造
11. 训练规模与 Hyperparameters
12. 评测体系：F1 + MBE
13. In-domain 实验结论
14. OOD 泛化与 Real-Web Ablation
15. Training Dynamics 与 Emergent Behaviors
16. 官方源码：Ray / veRL / Handler / Tool Server
17. 最值得沉淀的 Know-why
18. 最值得沉淀的 Know-how
19. DeepResearcher 的局限
20. 2026 后续研究如何修正它：DR Tulu / Search-R1++
21. 与 Search-R1 / WebThinker / Tongyi 的系统对比
22. 从零搭建 DeepResearcher-style MVP
23. Production Research RL 的改造建议
24. 来源、证据等级与阅读顺序

## 一句话

Search-R1 证明“模型可以用 RL 学会搜索”；DeepResearcher 进一步证明“如果训练环境不是真实 Web，学到的 Research Policy 会被环境上限卡住”。

DeepResearcher 总体架构：主策略端到端学习，浏览多 Agent 只是工具内部实现

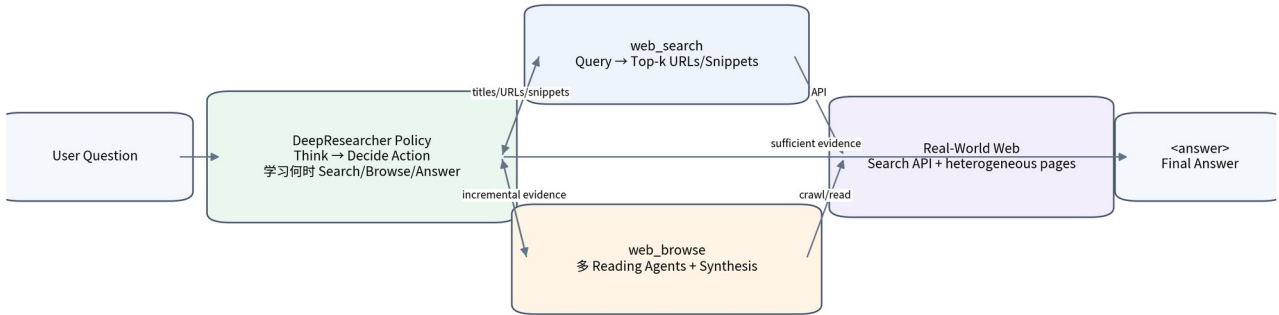


图 1 主策略端到端学习工具选择；Browsing Multi-Agent 被封装在 web\_browse 工具内部

# 1. 时代位置：从 Local RAG RL 到 Real-Web RL

2025 年初，Search-R1、R1-Searcher、ReSearch 等工作把 DeepSeek-R1 风格的 outcome-based RL 引入 “Reasoning + Search”。但主流做法仍然在**静态 Wikipedia / 本地向量库**上训练：检索语料固定、页面已经预处理、返回内容结构统一，搜索空间也被人为限定。

DeepResearcher 的研究问题是：**如果最终部署环境是开放 Web，为什么训练时却让 Agent 生活在一个干净、静态、可控的 RAG 世界？** 论文把这种 train-deploy mismatch 视为核心瓶颈：真实网页有噪声、时效变化、反爬、HTML 结构差异、API rate limit、网络失败与搜索质量波动，这些并不是“工程脏活”，而是研究策略必须学会处理的环境反馈。

Deep Research 训练范式演化：Local RAG RL → Real-Web RL → Long-form Research RL



图 2 DeepResearcher 的历史价值：把 RL Search Agent 的环境从固定语料迁移到真实 Internet

## Know-why

训练 Agent 时，\*\*环境不是一个中性的 I/O 适配层\*\*。环境决定模型能经历哪些失败、能学会哪些策略、以及推理时形成什么归纳偏置。

# 2. 核心命题：Environment 是 Research Policy 的一部分

DeepResearcher 的中心论点可以写成：

$$\text{Research Capability} \approx \text{Policy Learning} \times \text{Reward} \times \text{Environment Fidelity} \times \text{Tool Semantics} \times \text{Rollout Scale}$$

如果 Environment 是 Local RAG，模型可能学会的是“如何在一个高召回、干净的短 passage 集合里选答案”；如果 Environment 是 Live Web，模型才必须学习 Query reformulation、URL selection、网页深读、噪声过滤、交叉验证、失败恢复和停止。论文中“DeepResearcher (Local RAG)”对照组大幅掉分，正是在验证这一点。

## 可迁移结论

Agent Training 的 benchmark fidelity 不应只看“问题分布像不像线上”，还要看\*\*工具反馈分布、延迟、失败模式、噪声结构和可观测性\*\*是否接近线上。

### 3. Deep Research Trajectory : Think / Search / Browse / Answer

轨迹是一个受限 ReAct-like loop：模型必须先 reasoning，再决定动作。Reasoning 以 ``<think>`` 形式组织；搜索调用在真实实现中使用 JSON tool call；当信息足够时才输出 ``<answer>``。单个 rollout 最多 10 次 tool call。

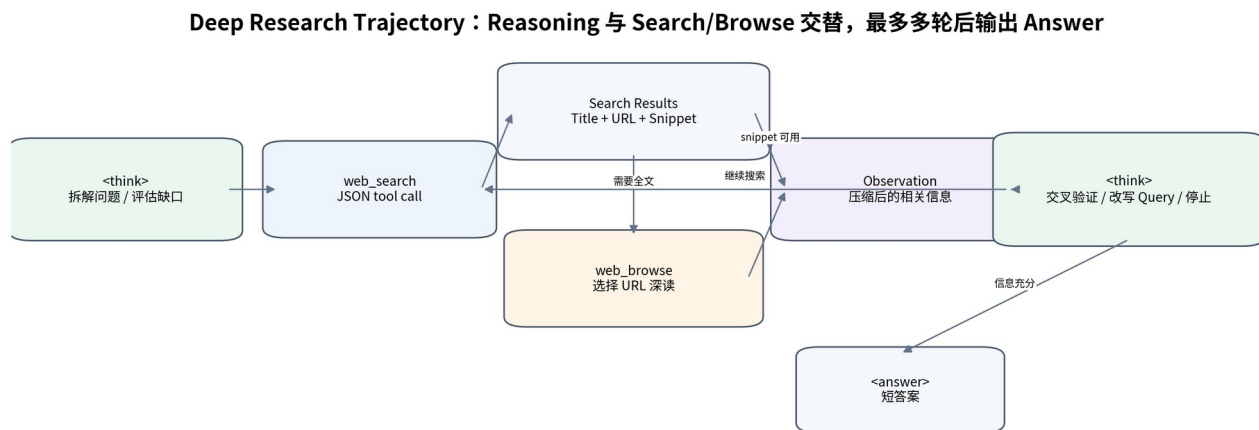


图 3 外层 policy 不需要人工写固定 workflow；它通过 RL 学习动作顺序与停止

这里和 prompt-engineered Deep Research 最大的不同是：论文没有预定义“先生成 5 个 subquestions，再并行搜索，再 rerank，再写报告”这种强流程。**策略只被给定动作空间和 outcome reward，搜索顺序从轨迹回报中学习。**

### 4. web\_search : 候选发现与 Query Policy

- 模型生成 JSON-formatted ``web_search`` 请求，参数是一个或多个 search queries。
- 搜索结果结构化返回 **title / URL / snippet**；论文当前使用固定 top-k（示例为 10）。
- Query 的“好坏”没有单独的人工 reward；它通过最终答案 F1 的 credit assignment 间接学习。
- 这意味着 Research Policy 必须自己学会：宽搜、改写、补充实体、交叉验证、以及何时停止重复搜索。

#### Know-how

如果最终目标是 end-to-end policy learning，尽量避免为 Query Planner 写过规则。规则可以放在 runtime 的安全与预算层，但\*\*策略层应保留足够自由度，让 reward 对搜索行为产生选择压力\*\*。

### 5. web\_browse : Reading Agents、Memory 与 Synthesis

搜索只返回候选信息，真正的难点是“读网页”。DeepResearcher 将浏览封装为 ``web_browse``：每个 query 有短期 memory；浏览器从网页前部开始按 segment 顺序读取，并根据当前问题、历史 memory 与新内容决定继续读下一段/URL，或停止。相关信息被持续追加到 memory，最后统一返回主 Agent。

web\_browse 内部多 Agent：并行读网页，按相关性增量写 Memory，再统一合成

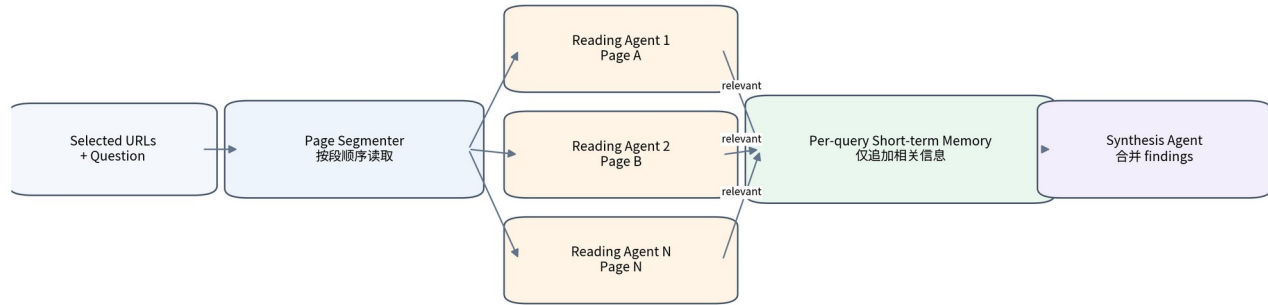


图 4 Search 负责“找候选”，Browse 负责“证据获取”；二者必须分离

## 6. 什工具部采用 Multi-Agent

长网页处理成本很高。论文采用多 Reading Agents 并行处理不同 URL，再由 Synthesis Agent 合并。一个重要细节是：作者强调 Multi-Agent 是 **web\_browse 的内部实现**，主 Research Policy 决定何时、为什么调用 Browse 仍然由 RL 学习。

因此这里的 Multi-Agent 不是“多角色协商”范式，而更接近**并行 I/O + Context Isolation + Map-Reduce Extraction**。这是一种非常值得复用的架构边界：把确定性的吞吐优化和信息压缩放在 Tool 内部，把不确定的搜索决策留给主策略。

### 架构原则

**\*\*Policy 负责 uncertain decisions; Tool Runtime 负责 deterministic throughput.\*\*** 不要因为内部用了多个 Agent，就把所有角色暴露给主模型。

# 7. 真实 Web RL 的三类工程难题

论文把真正让 Real-Web RL 难以落地的地方讲得很工程化，这也是 DeepResearcher 最有价值的部分之一。

| 挑战                 | 论文做法                                                                                  | 工程本质                                         |
|--------------------|---------------------------------------------------------------------------------------|----------------------------------------------|
| 高并发 Tool I/O       | 256 prompts × 16 rollouts，单个采样阶段可形成约 4096 个并发轨迹；使用 50-node CPU server cluster 分担搜索与抓取 | RL rollout 的瓶颈从 GPU compute 转向网络 I/O 与工具服务吞吐 |
| 反爬 / API 限流 / 网络失败 | Retry；相同 query 在预设窗口（示例 7 天）命中 Search Cache；适配搜索/LLM API rate limit                   | 环境必须可恢复，否则 reward noise 会被基础设施噪声污染           |
| 长网页信息抽取            | 网页分段；Reading Agents 并行；前部长期无关时提前跳过；Synthesis 汇总                                       | 把“页面读取”做成可增量停止的 budgeted process，而不是整页灌进上下文  |

## Know-why

真实 Web RL 最大的难点往往不是 PPO/GRPO 公式，而是\*\*每个 rollout 都会调用真实外部系统\*\*。如果工具失败率高、延迟不稳定或结果不可复现，优化器看到的 reward 就会变成高噪声信号。

# 8. GRPO、Observation Masking 与训练目标

DeepResearcher 使用 GRPO：每个 prompt 生成一组 rollouts，用组内相对回报估计 advantage，无需单独训练 critic。官方训练脚本显示 backbone 为 Qwen2.5-7B-Instruct，actor learning rate 1e-6，`agent\_grpo.n=16`，`max\_turns=10`，并启用 KL loss。

RL 训练主循环：大规模实时 Web rollout + observation masking + outcome-only GRPO

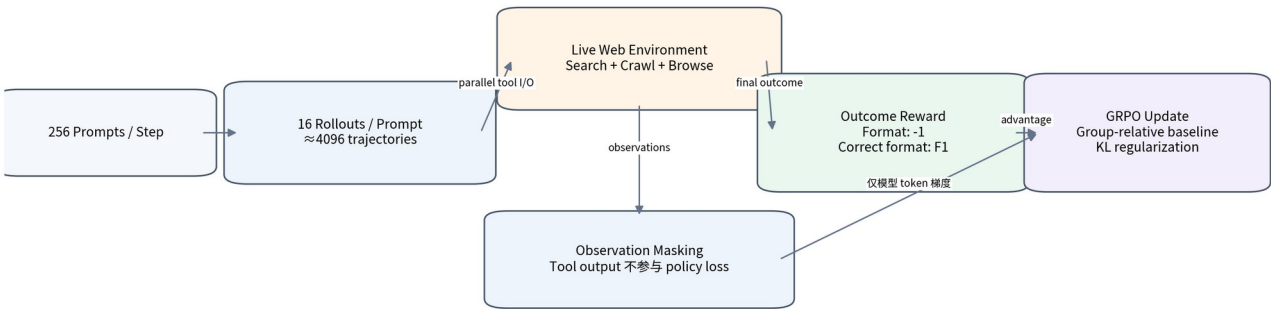


图 5 GRPO 不是最关键的创新；关键是它被接到了真实 Web 的大规模 rollout 上

论文特别强调 **Masking Observations**：Tool output 是环境 observation，而不是模型应该生成的目标 token，因此 observation token 不参与 policy loss。只让模型自己生成的 reasoning / tool call / answer token 接收梯度。

## Know-how

Tool-augmented RL 里最容易被忽视的是 loss mask。若把外部 observation 也当作 teacher token 学习，会把环境内容错误地纳入生成目标，破坏 policy optimization 的语义。

## 9. Reward：为什么采用 Format Penalty + F1

训练数据来自带短答案 ground truth 的开放域 QA，因此 reward 极简：格式错误直接 -1；格式正确时使用 word-level F1。没有 search-step reward、citation reward、source quality reward 或人工 planning reward。

```
reward = -1 if output format is invalid
reward = word_level_F1(answer, gt) otherwise
```

这种 outcome-only reward 的优点是少人为 shaping，更容易观察 “planning / cross-validation / reflection” 是否自然涌现；缺点也很明确：它只适合**可验证短答案**，并不能直接优化长篇研究报告的完整性、来源质量、引用忠实度与洞察结构。

### 关键边界

DeepResearcher 应被理解为 **\*\*Deep Search / Research QA Agent\*\*** 的重要训练范式，而不是 2026 意义上已经完成的 Long-form Deep Research Report Model。



# 10. Search-dependent Data：80K 数据如何构造

论文真正聪明的数据设计不是构造大量人工 trajectory，而是先保证问题“必须搜索”。训练源来自 NQ、TriviaQA、HotpotQA、2WikiMultiHopQA。

训练数据核心不是“多”，而是 Search-dependent：主动剔除模型本来就会的题

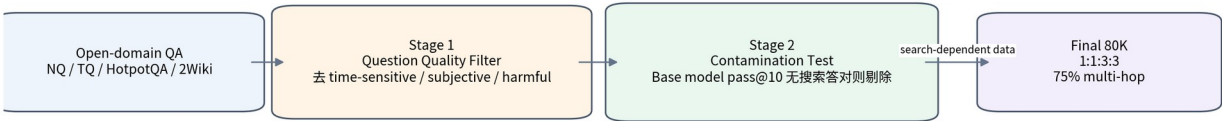


图 6 先过滤“模型无需搜索就能答”的样本，才有可能真正训练 Search Policy

两阶段过滤：第一阶段用 DeepSeek-R1 对题目质量做分类，剔除时效性强、主观或潜在有害的问题；第二阶段做 contamination detection——对每道候选题让训练所用 base model 无搜索随机采样 10 次，只要 pass@10 中出现 ground truth，就剔除该题。

最终得到 80,000 样本，NQ:TQ:HotpotQA:2Wiki = 1:1:3:3，即 75% 是 multi-hop。这里的数据工程目标不是“模仿正确搜索轨迹”，而是构造足够多的 knowledge gap，让 RL 必须通过工具交互获得回报。

### Know-why

训练 Tool Use 的第一原则不是“有没有 tool-call label”，而是\*\*没有工具时是否真的存在不可跨越的信息缺口\*\*。如果模型靠参数知识就能答，训练会学成“装作搜索”。

# 11. 训练规模与 Hyperparameters

| 项目                                   | 论文 / 官方脚本设置                                    |
|--------------------------------------|------------------------------------------------|
| Backbone                             | Qwen2.5-7B-Instruct                            |
| RL Framework                         | veRL，GRPO 风格 agent rollout                     |
| Prompts / step                       | 256                                            |
| Rollouts / prompt                    | 16                                             |
| Nominal trajectories / rollout stage | 4096                                           |
| Max tool calls / rollout             | 10                                             |
| Mini-batch                           | 4096                                           |
| Official script max model len        | 32768                                          |
| Actor LR                             | 1e-6                                           |
| KL coef                              | 0.001                                          |
| Runtime                              | Ray + distributed tool handler + online search |

需要注意：论文叙述和代码参数存在实现版本差异的可能性，例如 paper 里把 GRPO 描述为 group-relative policy optimization，而官方脚本入口仍调用 `verl.trainer.main\_ppo` 并挂载 `agent\_grpo` 参数。这更像是在 veRL PPO trainer 上实现 GRPO-style agent rollout，而不是一个完全独立的训练器。

## 12. 评测体系：F1 + MBE

评测分为 in-domain 与 out-of-domain。ID：NQ、TQ、HotpotQA、2Wiki；OOD：MuSiQue、Bamboogle、PopQA。除 Bamboogle 全量 125 个样本外，其余 dev 集随机采样 512。

除了 token-level F1，论文还用 Model-Based Evaluation (MBE)：让 GPT-4o-mini 根据 question、ground truth 与模型答案判断 correct / incorrect，再计算 accuracy。论文认为 MBE 对“语义正确但表面表达不同”的情况更可靠，因此主要结论更多看 MBE。

### 评测提醒

F1 与 MBE 都还是 answer-centric。它们无法评估 citation faithfulness、evidence diversity、source freshness、research coverage 或“诚实拒答”的价值。后续 Long-form DR benchmark 必须补这些维度。

# 13. In-domain 实验结论

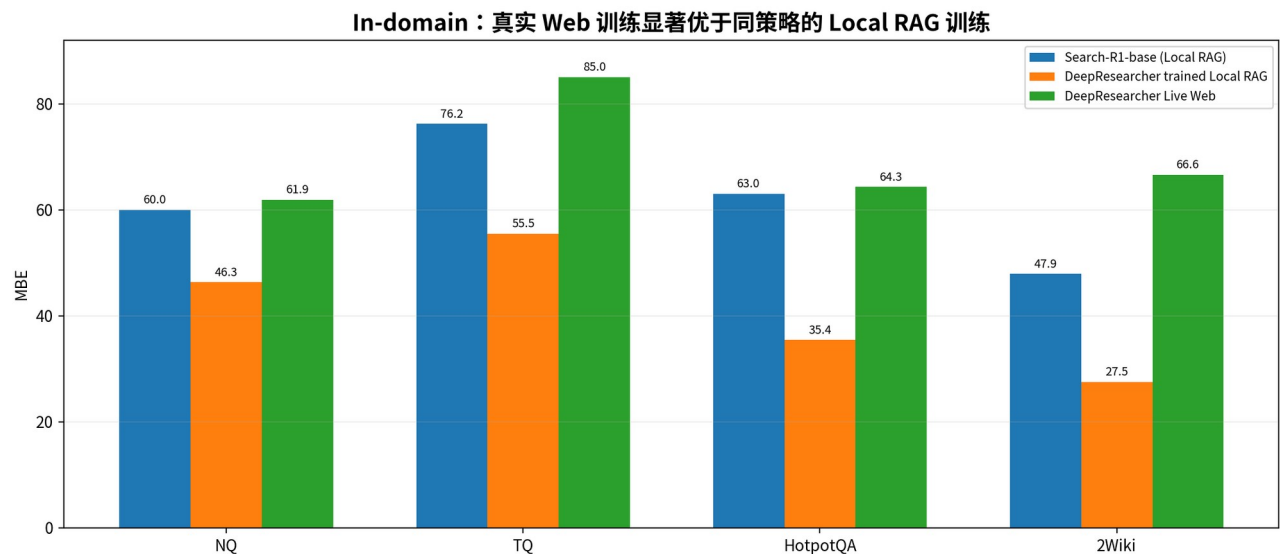


图 7 MBE: DeepResearcher 在四个 ID 数据集都显著优于同模型 Local-RAG 训练版本

| Method / Env                | NQ F1/MBE   | TQ F1/MBE   | Hotpot F1/MBE | 2Wiki F1/MBE |
|-----------------------------|-------------|-------------|---------------|--------------|
| Search-R1-base / Local RAG  | 45.4 / 60.0 | 71.9 / 76.2 | 55.9 / 63.0   | 44.6 / 47.9  |
| R1-Searcher / Web Search    | 35.4 / 52.3 | 73.1 / 79.1 | 44.8 / 53.1   | 59.4 / 65.8  |
| DeepResearcher (Local RAG)  | 29.5 / 46.3 | 51.9 / 55.5 | 29.4 / 35.4   | 26.3 / 27.5  |
| DeepResearcher / Web Search | 39.6 / 61.9 | 78.4 / 85.0 | 52.8 / 64.3   | 59.7 / 66.6  |

最值得注意的不是 DeepResearcher 在每个 F1 都绝对第一，而是：同一 DeepResearcher 训练范式一旦改成 Local RAG，性能全面坍塌。这比和别的 baseline 比较更能支持作者的中心命题。

# 14. OOD 泛化与 Real-Web Ablation

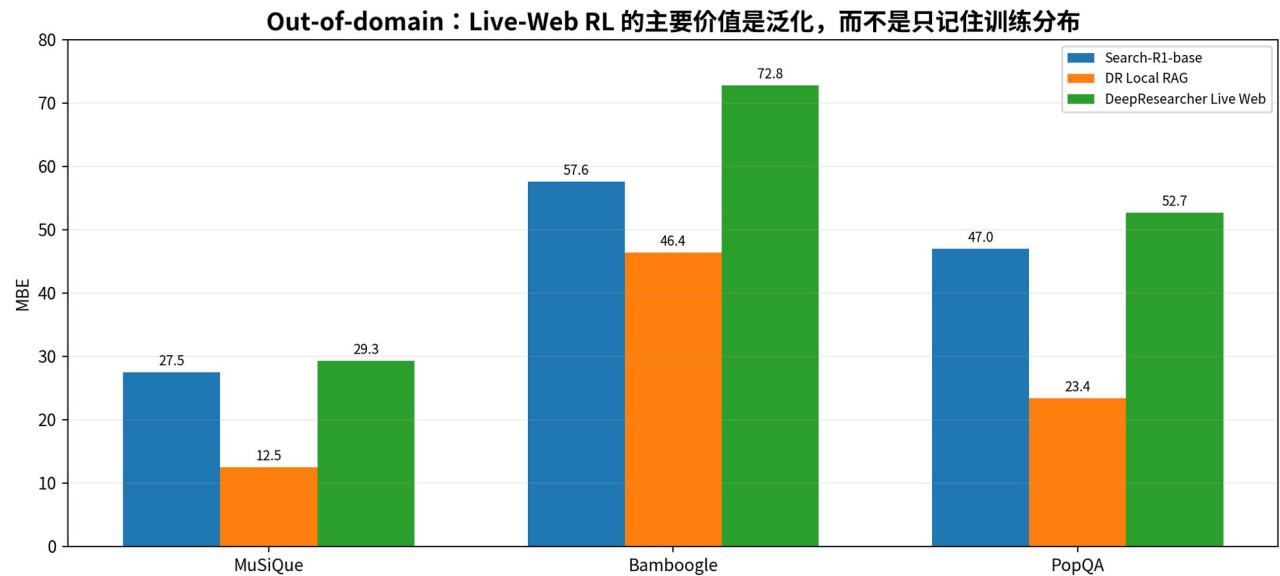


图 8 OOD MBE: MuSiQue / Bamboogle / PopQA 都显示 Live-Web 训练的泛化优势

| Method / Env                | MuSiQue F1/MBE | Bamboogle F1/MBE | PopQA F1/MBE |
|-----------------------------|----------------|------------------|--------------|
| Search-R1-base / Local RAG  | 26.7 / 27.5    | 56.5 / 57.6      | 43.2 / 47.0  |
| R1-Searcher / Web Search    | 22.8 / 25.6    | 64.8 / 65.6      | 42.7 / 43.4  |
| DeepResearcher (Local RAG)  | 12.7 / 12.5    | 42.7 / 46.4      | 23.2 / 23.4  |
| DeepResearcher / Web Search | 27.1 / 29.3    | 71.0 / 72.8      | 48.5 / 52.7  |

Bamboogle 尤其有意义，因为其问题并不完全依赖 Wikipedia，能更直接测试“是否真的学会在开放 Web 里找信息”。DeepResearcher 的 72.8 MBE 明显高于 Search-R1-base 的 57.6 与 R1-Searcher 的 65.6。

核心实验结论

如果只记一个 ablation: \*\*DeepResearcher Live Web >> DeepResearcher Local RAG\*\*。这是整篇论文最强的 know-why 证据。

## 15. Training Dynamics 与 Emergent Behaviors

论文观察到随着 RL step 增长，F1 从约 0.375 上升到约 0.55；同时复杂问题所需 tool calls 与 response length 也上升。特别是更高 hop 的问题持续增加搜索轮数，说明模型不是简单学会一个固定长度模板，而是在把额外 compute 分配给更难的问题。

Case Study 总结出四类自然涌现行为：

- **Planning**：面对 multi-hop 问题先形成步骤，并在过程中动态合并或调整。
- **Cross-validation**：即使第一次搜索已经得到正确候选，也会继续搜索进行验证。
- **Reflection / Redirection**：当 observation 与预期不一致时，主动改写查询，避免 stuck。
- **Honesty**：当没有找到足够证据时，能承认无法确定，而不是强行猜测。

### 为什么重要

这些行为没有通过 SFT planning trace 或专门的“reflection reward”逐项监督。论文把它们视作 outcome-only RL 在真实环境里的 emergent strategy。

## 16. 官方源码：Ray / veRL / Handler / Tool Server

官方仓库不是一个轻量 inference demo，而是一套面向训练的分布式系统。README 的启动顺序是：先启动 Ray；再启动 server handler（搜索、浏览和辅助 LLM）；配置 `server\_url\_list`；启动 handler；最后运行 `train\_grpo.sh`。

```
Ray cluster
├─ veRL actor / rollout workers (GPU)
│   └─ agent rollout emits tool requests
│       └─ handler (routing / batching)
│           ├── search API
│           ├── crawler / browse readers
│           └─ auxiliary LLM / synthesis
│               ↓
│           observations → rollout → reward → GRPO
```

这套结构透露一个关键工程事实：**Research RL 是异构系统训练**。GPU 负责 policy generation / backward，CPU + network cluster 负责高并发环境交互；两边如果没有解耦，GPU 会大量空等 I/O。

### Production know-how

把 Tool Environment 做成独立服务层，并支持 cache / retry / concurrency control / backpressure；不要把 crawl/search 直接写进 rollout worker 里。

# 17. 最得淀的 Know-why

- **Environment Fidelity > Offline Convenience**：如果部署在 live web，训练最好也暴露真实搜索噪声与失败模式。
- **Search-dependent Data > Tool-call Demonstrations**：只有当题目无法靠参数知识解决，模型才有动机学搜索。
- **Tool Semantics 决定策略空间**：Search 发现候选，Browse 获取证据；把两者合并会削弱模型对“发现 vs 阅读”的决策能力。
- **Outcome-only RL 能发现策略，但 reward scope 决定能力上限**：短答案 F1 可以训练搜索问答，却无法直接训练高质量长报告。
- **Multi-Agent 的价值可以只是 Runtime Parallelism**：不必把每个子代理变成显式社会角色。

# 18. 最值得沉淀的 Know-how

| 主题          | 可直接复用的做法                                                              |
|-------------|-----------------------------------------------------------------------|
| 数据          | Base-model pass@k contamination filter：先证明“无工具不会”，再把题放进 tool-RL       |
| Environment | 搜索、抓取、页面阅读分层；失败有 retry；search query 做 TTL cache                       |
| Rollout     | 大 batch × 多 samples 时把工具服务独立扩容；GPU 不等待单线程 crawler                     |
| Loss        | Tool observations 必须 mask；policy loss 只覆盖模型动作 token                   |
| 浏览          | 长页面分段、按需继续、早停；多 URL 并行 reader + synthesis                             |
| 评测          | 至少同时看 exact/F1 与 semantic judge；并额外统计 tool cost / failures / coverage |
| 对照实验        | 同 policy / 同 reward，仅替换 Environment，直接测环境真实性的增益                       |

## 可以沉淀成一个设计公式

Agent RL = \*\*Search-dependent Task Distribution × Realistic Environment × Learnable Action Space × Correct Loss Mask × Stable Outcome Signal × High-throughput Rollout Runtime\*\*。

## 19. DeepResearcher 的局限

- **Reward 与任务错位**：word-level F1 适合短答案，不适合开放式长报告。论文自己明确把更复杂 reward 留作未来工作。
- **模型规模有限**：实验只验证 Qwen2.5-7B-Instruct，未验证更大模型的 scaling behavior。
- **Web 非平稳**：live search 结果会随时间、地区、搜索引擎策略变化，复现实验天然更难。
- **Browser 内部仍有人工设计**：例如从页面头部顺序读、前部长无关则跳过；主策略端到端，但 environment tool 本身仍包含 heuristic。
- **Citation / provenance 不是主要目标**：实验只评 answer correctness，没有 claim-level evidence verification。
- **成本与合规**：大量真实搜索/抓取意味着 API 成本、反爬策略、隐私/版权和服务条款约束。

## 20. 2026 后续研究如何修正它：DR Tulu / Search-R1++

DeepResearcher 的历史意义很大，但 2026 的后续研究也恰好暴露了它的边界。DR Tulu 直接指出：多数开放 Deep Research 模型以 RLVR 在可验证短答案上训练，**无法自然扩展到 long-form research**；它因此提出 Reinforcement Learning with Evolving Rubrics (RLER)，让 rubric 随 policy 和新检索证据共同演化，用于长报告的 on-policy 评价。

另一条 2026 研究线 Search-R1++ 对 Search-R1 类 RL 做系统实证：其报告显示 F1 reward 在某些 prompt/template 组合下可能出现 answer-avoidance / training collapse，加入 action-level penalty 可缓解；同时不同 policy optimization 方法的稳定性差异明显。这个结果提醒我们：DeepResearcher 的“F1 + GRPO”应被看作 2025 年有效的具体配方，而不是普适最优解。

### 演化逻辑

DeepResearcher 解决的是 **\*\*Where to train? — Real Web\*\***；DR Tulu 进一步解决 **\*\*What to reward? — Long-form evolving rubrics\*\***。两者合起来才接近完整的现代 Deep Research RL。

## 21. 与 Search-R1 / WebThinker / Tongyi 的系统对比

DeepResearcher 的历史位置：真正关键的桥梁是“把真实 Web 变成 RL Environment”

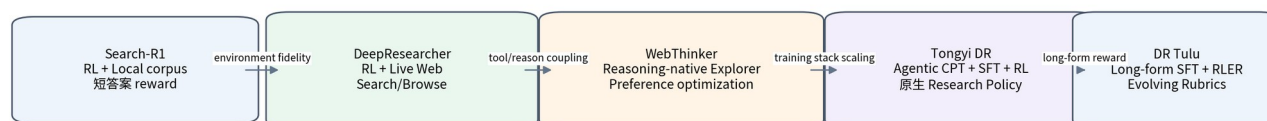


图9 DeepResearcher 是从“静态 RAG RL”走向“真实 Research Environment”的关键中间站

| 系统                  | Policy 学习                                  | 环境                            | 任务输出                 | 主要创新                                       |
|---------------------|--------------------------------------------|-------------------------------|----------------------|--------------------------------------------|
| Search-R1           | RL                                         | Local RAG / retriever         | 短答案                  | Reasoning + Search interleave              |
| DeepResearcher      | GRPO-style RL                              | **Live Web + Browse**         | 短答案                  | Environment fidelity + real-world rollout  |
| WebThinker          | Preference optimization / reasoning policy | Live Web + Deep Explorer      | 答案 + 报告              | Search/Click/Draft 进入 reasoning trajectory |
| Tongyi DeepResearch | Agentic CPT + SFT + RL                     | 多种搜索/工具环境                     | 研究答案/报告              | Research capability 内化到模型训练全栈              |
| DR Tulu             | SFT + RLER                                 | MCP multi-tool research stack | **Long-form report** | Evolving rubrics for open-ended research   |



## 22. 从零搭建 DeepResearcher-style MVP

如果目标是复现这篇工作的核心，而不是照搬其集群规模，建议按下面顺序构建：

- **Step 1 — Search-dependent dataset**：从有 ground truth 的 multi-hop QA 开始；对 base model 做 no-tool pass@k 筛选。
- **Step 2 — Minimal action space**：只提供 ``search(query[])``、``browse(urls, question)``、``answer(text)``；不要一开始就加几十个工具。
- **Step 3 — Browser service**：页面抽取、分段、relevance memory、early stop、并行 URL readers；统一返回短 evidence bundle。
- **Step 4 — Observation mask**：在 tokenizer/action protocol 层明确哪些 token 是 policy output，哪些是 environment observation。
- **Step 5 — Outcome reward**：短答案先用 EM/F1 + format；将 action cost 记录为诊断指标，不急着 reward shaping。
- **Step 6 — Async rollout**：policy generation 与 tool execution 解耦；工具服务支持 queue / retry / TTL cache。
- **Step 7 — Environment ablation**：同一训练设置分别接 Local RAG 与 Live Web，测 OOD 泛化差异。
- **Step 8 — Long-form upgrade**：再引入 citation verifier、rubric judge、coverage / faithfulness、evolving rubrics。

```
for question in batch:
    trajectories = rollout(policy, n=G, env=live_web)
    rewards = evaluate_final_answer(trajectories)
    mask = policy_tokens_only(trajectories)
    update_with_group_relative_advantage(policy, trajectories, rewards, mask)
```

**不要先做的事**  
不要一开始构建复杂的 Planner/Reviewer/Writer 角色图，也不要先生成大量 imitation trajectories。  
DeepResearcher 的核心实验价值来自 “\*\*让 outcome signal 在真实环境中选择策略\*\*”。

## 23. Production Research RL 的改造建议

| 问题      | 论文方案                      | 面向生产的下一步                                                                           |
|---------|---------------------------|------------------------------------------------------------------------------------|
| Web 不稳定 | Retry + 7-day query cache | 版本化 observation；记录 search provider / timestamp / locale；支持 deterministic replay    |
| 奖励短视    | F1 + format               | 长报告使用 claim-level rubric + citation faithfulness + coverage + cost                 |
| 工具成本    | 大规模实时 API                 | tool budget / domain quota / cache hit-rate / cost-aware reward diagnostics        |
| 页面质量    | Reading Agent early stop  | source-quality prior、anti-SEO、duplicate clustering、conflict detection              |
| 复现困难    | Live search               | 训练用 snapshot + 部分 live mix；评测使用 frozen web / RetroSearch 类环境                       |
| 可观测性    | rollout logs              | trace<br>schema: thought/action/observation/source/cost/latency/error/reward 全量结构化 |

**面向 2026 的组合范式**  
更完整的 Research Agent Training 可以写成：\*\*DeepResearcher 的 Real-Web Environment + Tongyi 的多阶段 Agent Training + DR Tulu 的 Long-form Reward + DeerFlow/Deep Agents 类 Runtime Guardrails\*\*。

# 24. 来源、证据等级与阅读顺序

## A 级：论文 / 官方源码

- Zheng et al. DeepResearcher: Scaling Deep Research via Reinforcement Learning in Real-world Environments. EMNLP 2025. ACL Anthology / arXiv:2504.03160.
- GAIR-NLP/DeepResearcher 官方 GitHub: README、train\_grpo.sh、src/handler、src/llm\_agent 等源码。
- GAIR/DeepResearcher-7b 官方 Hugging Face Model Card: Qwen2.5-7B-Instruct base, Apache-2.0。
- Jin et al. Search-R1: Training LLMs to Reason and Leverage Search Engines with Reinforcement Learning. arXiv:2503.09516.
- Shao et al. DR Tulu: Reinforcement Learning with Evolving Rubrics for Deep Research. arXiv:2511.19399 v3 / ICML 2026; Ai2 官方博客。
- Xu et al. How to Train Your Deep Research Agent? Prompt, Reward, and Policy Optimization in Search-R1. arXiv:2602.19526.

## B 级：本报告综合推论

- “Environment Fidelity 是 Agent Capability 的训练变量” 是对论文 Real-Web vs Local-RAG ablation 的工程抽象。
- “Policy 负责 uncertain decisions; Runtime 负责 deterministic throughput” 是对主策略与 web\_browse 内部 Multi-Agent 边界的架构提炼。
- “DeepResearcher 解决 Where to train, DR Tulu 解决 What to reward” 是跨论文的演化总结，不是任何单篇论文的原句。

## 建议阅读顺序

1) 先看论文 Figure 1 + Section 3.1/3.2, 理解轨迹和真实 Web 工程; 2) 看 Table 1/2 与 Local-RAG ablation; 3) 看 Appendix A 的 contamination filtering; 4) 打开官方 `train\_grpo.sh` 对照论文参数; 5) 再读 Search-R1, 理解 DeepResearcher 的前置范式; 6) 最后读 DR Tulu 与 Search-R1++, 理解短答案 F1/GRPO 之后为什么需要新 reward 与训练稳定性研究。

### 最终结论

DeepResearcher 最值得留下的不是某个具体 GRPO 超参，而是一条更长期有效的工程原则：\*\*如果你希望 Agent 在真实世界里学会研究，就必须让“真实世界的复杂性”进入训练闭环，而不是只在推理时才暴露给它。  
\*\*